UNIVERSIDAD DON BOSCO	**UNIVERSIDAD DON BOSCO FACULTAD DE INGENIERÍA ESCUELA DE COMPUTACIÓN**	**CICLO 02-2026**
	MATERIA: **DESARROLLO DE SOFTWARE EMPRESARIAL**	**GUIA DE LABORATORIO #6**
	PRÁCTICA: **INTRODUCCION A DOCKER**	

I. OBJETIVOS

- Que el estudiante aprenda a instalador Docker.
- Que el estudiante aprenda a desplegar una aplicación en un contenedor.

II. INTRODUCCIÓN TEORICA

Docker

Docker es una plataforma de software que permite crear, desplegar y gestionar aplicaciones en contenedores, donde, un contenedor es una unidad estandarizada de software que empaqueta el código y todas sus dependencias, de modo que la aplicación se puede ejecutar de manera rápida y confiable en cualquier entorno.

Características

- **Contenedores ligeros**: a diferencia de las máquinas virtuales, los contenedores comparten el mismo núcleo del sistema operativo, lo que los hace más ligeros y rápidos.
- **Portabilidad**: los contenedores Docker pueden ejecutarse en cualquier lugar: en la máquina local, en servidores on-premise, en la nube, etc., sin preocuparse por las diferencias en las configuraciones de las máquinas.
- **Aislamiento**: cada contenedor se ejecuta de manera aislada, lo que significa que los procesos dentro de un contenedor no afectan a los que están fuera de él, lo que mejora la seguridad y la estabilidad.
- **Gestión de imágenes**: Docker utiliza imágenes para empaquetar aplicaciones. Estas imágenes son versiones inmutables que se pueden compartir y reutilizar, lo que facilita la colaboración y el despliegue.
- **Eficiencia de recursos**: Como los contenedores comparten el mismo núcleo, requieren menos recursos que las máquinas virtuales.

Usos de Docker

- **Desarrollo de software**: Docker permite a los desarrolladores crear entornos de desarrollo consistentes, lo que facilita la replicación de entornos de producción en máquinas locales.
- **Despliegue continuo**: Con Docker, se puede automatizar la integración y el despliegue continuo (CI/CD), facilitando la entrega rápida y confiable de aplicaciones.
- **Microservicios**: Docker es ideal para arquitecturas de microservicios, donde cada servicio puede ejecutarse en su propio contenedor.
- **Pruebas automatizadas**: Docker permite ejecutar pruebas en un entorno controlado, lo que reduce las "fallas por configuración" que podrían surgir en diferentes entornos.
- **Computación en la nube**: Muchas plataformas en la nube soportan Docker, lo que facilita el despliegue de aplicaciones en diferentes proveedores de nube.

Ventajas

- **Velocidad**: El inicio de un contenedor Docker es mucho más rápido que el de una máquina virtual, lo que acelera el despliegue y desarrollo.
- **Consistencia**: Docker asegura que una aplicación se ejecute de la misma manera en cualquier entorno, lo que reduce problemas de "funciona en mi máquina".

- **Escalabilidad**: Docker facilita la escalabilidad horizontal de aplicaciones mediante la distribución de contenedores en múltiples instancias.
- **Facilidad de uso**: Docker tiene una curva de aprendizaje relativamente baja y cuenta con una amplia comunidad y documentación.

Dockerfile

El Dockerfile es un archivo de texto que contiene un conjunto de instrucciones para construir una imagen Docker. Esencialmente, define el entorno en el que una aplicación se ejecutará, incluyendo:

- **Imagen base**: Establece el sistema operativo y otras herramientas o lenguajes necesarios para la aplicación. Por ejemplo, una imagen base de .NET, Node.js, o Python.
- **Instrucciones para construir la aplicación**: Incluye pasos como copiar el código fuente, instalar dependencias, compilar la aplicación, etc.
- **Configuración de ejecución**: Define cómo se ejecutará la aplicación cuando el contenedor se inicie (p. ej., el comando ENTRYPOINT).

```
# Imagen base
FROM ubuntu:24.04

# Información del mantenedor (opcional)
LABEL maintainer="ejemplo@correo.com"

# Actualizar paquetes e instalar curl
RUN apt-get update && \
    apt-get install -y curl && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*

# Directorio de trabajo
WORKDIR /app

# Copiar archivos locales al contenedor
COPY . /app

# Variable de entorno
ENV APP_ENV=development

# Puerto expuesto
EXPOSE 8080

# Comando por defecto
CMD ["bash", "-c", "echo 'Contenedor iniciado'; sleep infinity"]
```

docker-compose.yml

El archivo docker-compose.yml es un archivo de configuración utilizado por Docker Compose, una herramienta que permite definir y ejecutar aplicaciones multi-contenedor. Este archivo describe los servicios (contenedores) que componen una aplicación, así como la configuración para cada uno de ellos.

Algunas de las configuraciones que puedes definir en docker-compose.yml incluyen:

- **Servicios**: Cada servicio corresponde a un contenedor que forma parte de la aplicación, como una API, una base de datos, un servidor web, etc.
- **Imágenes y construcción**: Puedes especificar si un servicio debe usar una imagen existente o construir una nueva a partir de un Dockerfile.
- **Puertos**: Define cómo los puertos internos de los contenedores se mapearán a los puertos del host.
- **Volúmenes**: Permite compartir datos entre el contenedor y el sistema de archivos del host, útil para persistir datos o compartir configuraciones.
- **Variables de entorno**: Puedes configurar variables de entorno para cada servicio, como cadenas de conexión a bases de datos o claves API.
- **Dependencias**: Especifica qué servicios deben estar activos antes de que otro servicio pueda arrancar

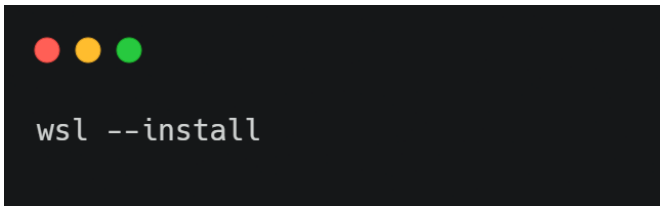
III. DESARROLLO DE LA PRÁCTICA

Parte 1 – Instalación de Docker en Windows

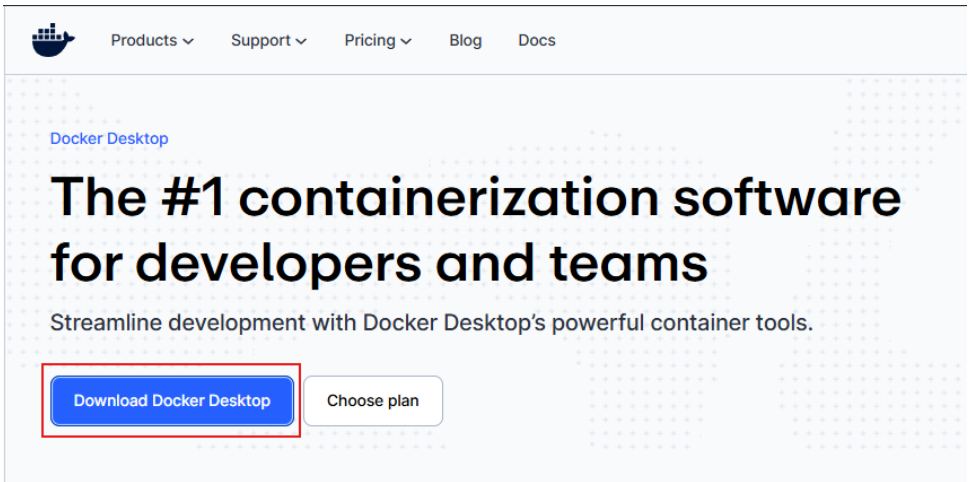
Antes de iniciar la instalación verifique los requisitos del sistema:

- **Sistema Operativo**: Windows 11.
- **Virtualización**: se debe habilitar la virtualización en la BIOS de su computadora. Docker para Windows requiere la funcionalidad de virtualización para Hyper-V o WSL 2 (Windows Subsystem for Linux 2).
- **RAM**: Mínimo 4 GB de RAM (se recomienda 8 GB o más).

1. Abra una nueva terminal de PowerShell en su equipo y escriba el siguiente comando.

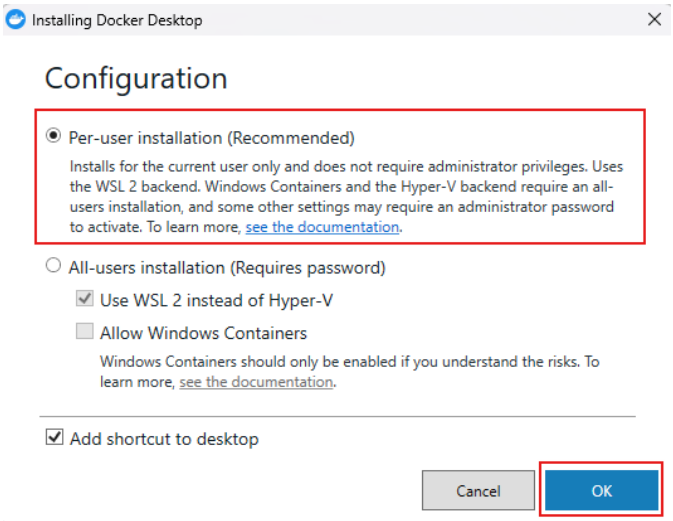


2. Ingrese al sitio https://www.docker.com/products/docker-desktop/ y descargue el instalador de Docker Desktop correspondiente a su sistema operativo.

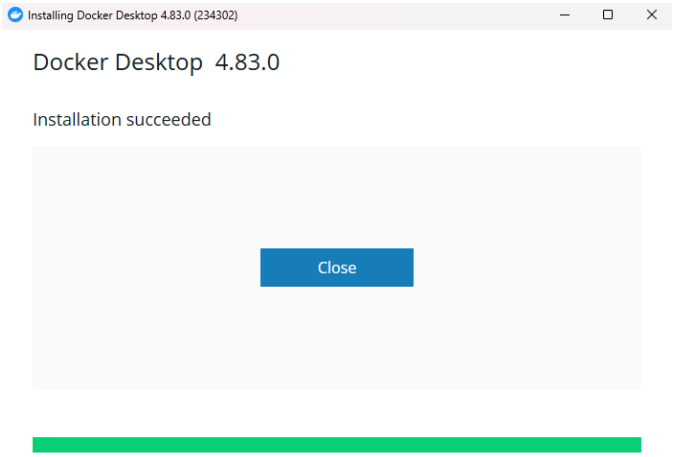


3. Una vez finalizada la descarga ejecute el instalador y siga las instrucciones a continuación.

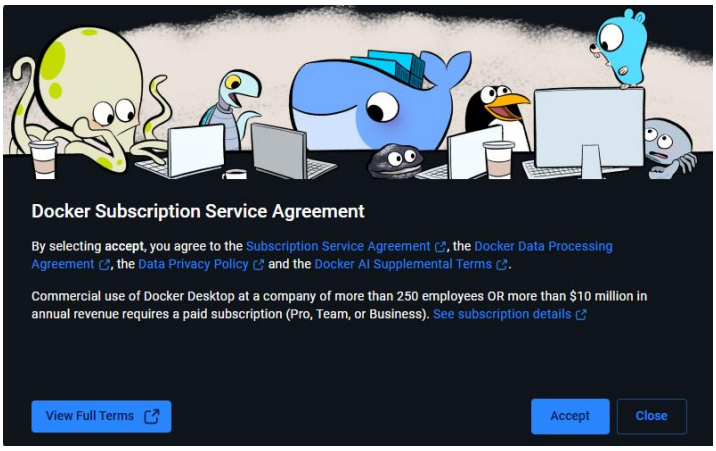
4. Seleccione la instalación **para un solo usuario** y haga clic en **Ok.** Opcionalmente puede elegir la opción de agregar el acceso directo al escritorio**.**



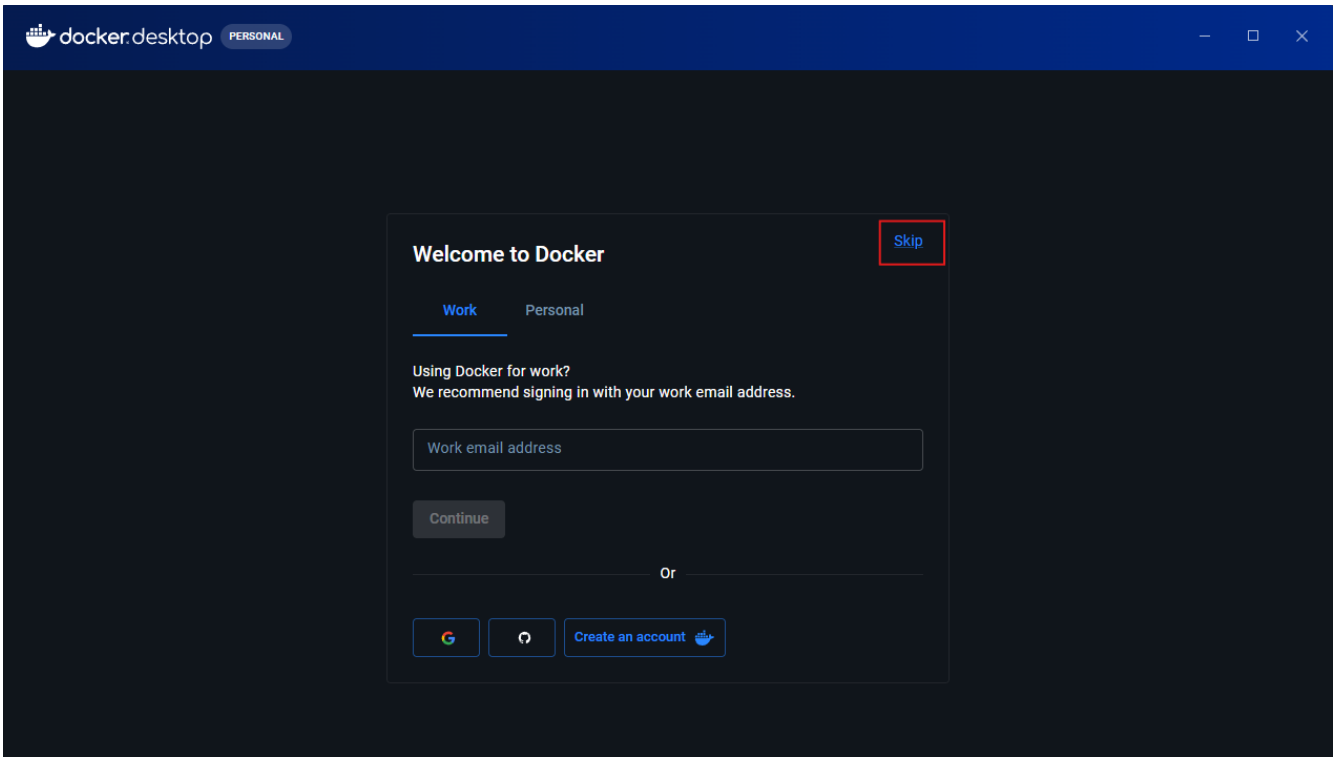
5. Espere que finalice la instalación y haga clic en **Close**.



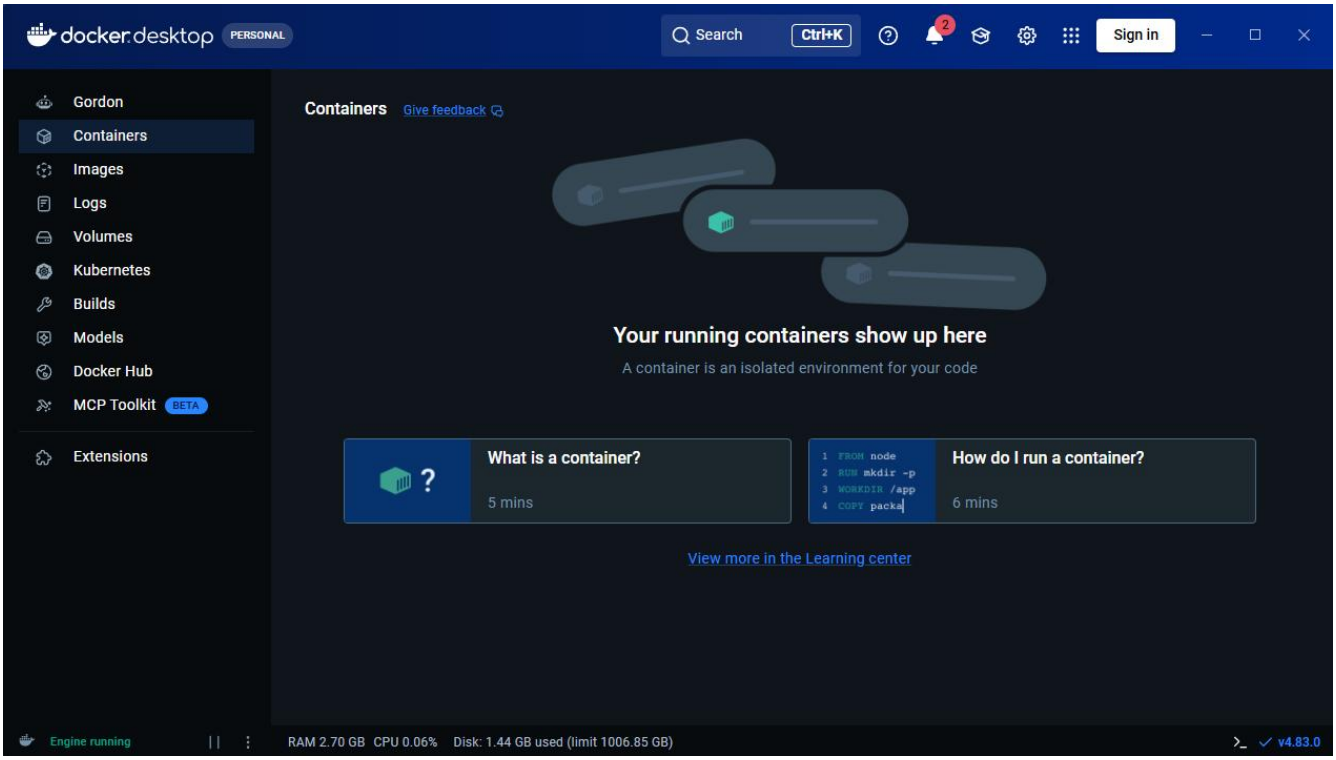
6. Inicie la aplicación Docker Desktop, si es la primera vez que se ejecuta, acepte los términos de servicio y haga clic en **Aceptar**.



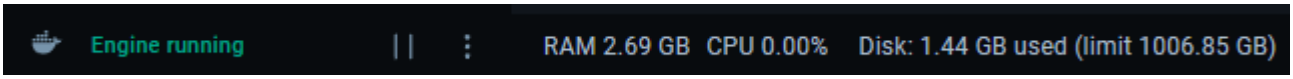
7. Omita el paso de creación de cuenta en Docker Hub haciendo clic en **Skip**.



8. Si todo ha ido bien, deberá ver una UI como la de la siguiente ilustración.

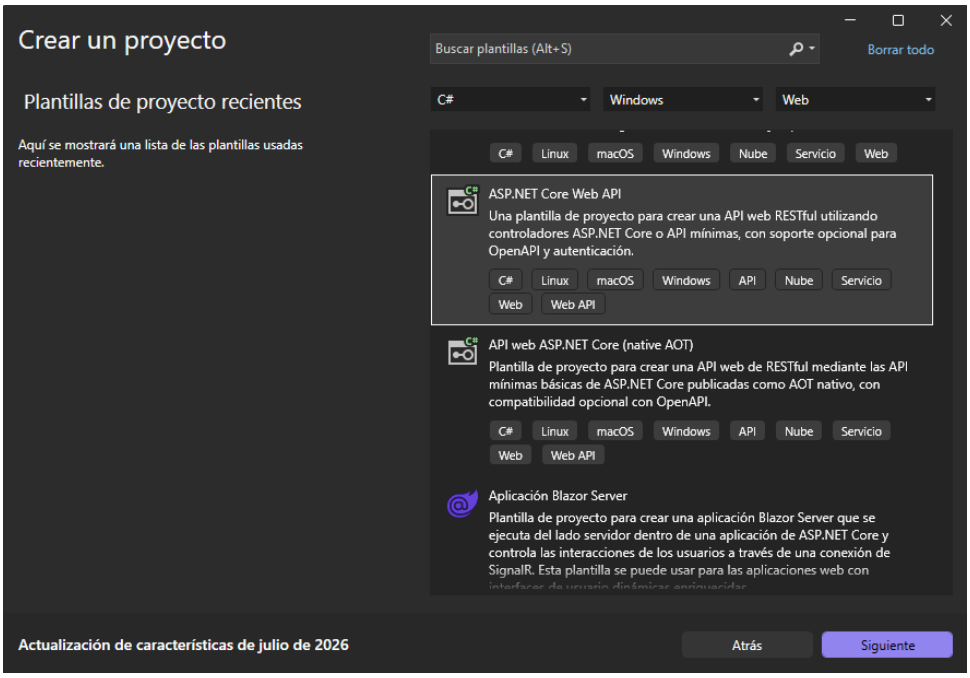


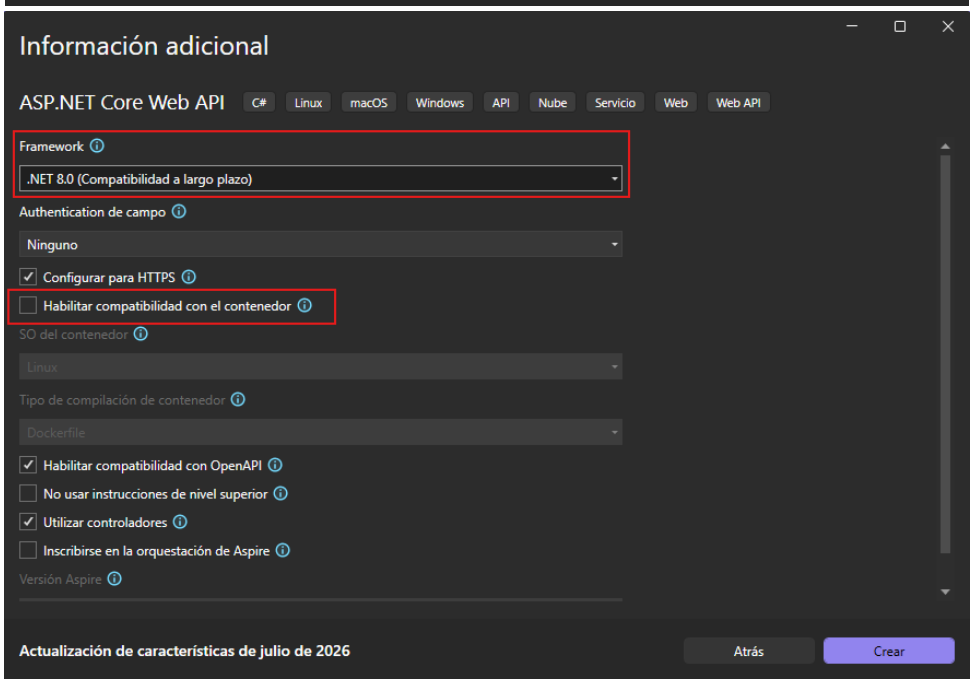
9. Compruebe el estado del motor de Docker en la esquina inferior izquierda, debe mostrar **Engine running**.



Parte 2 – Implementando un WEB API

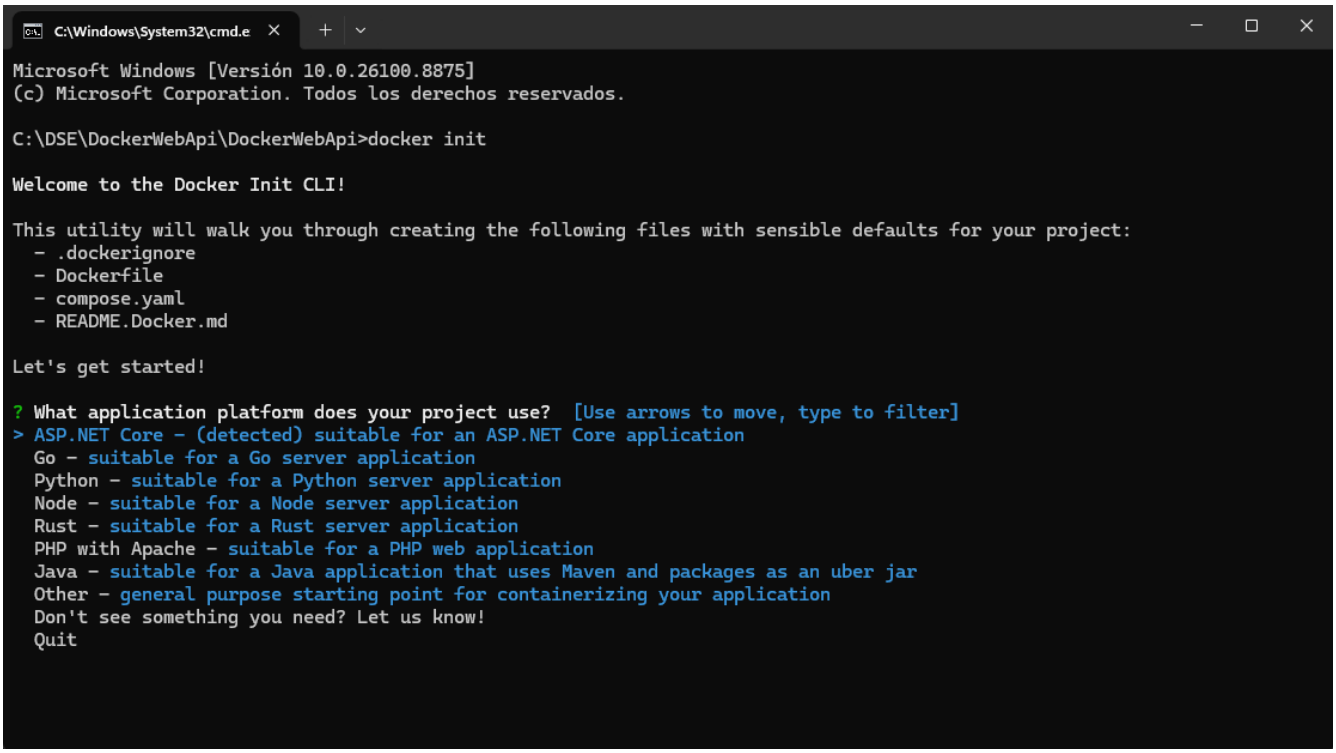
1. Cree un nuevo proyecto en Visual Studio con el nombre **DockerWebApi**, utilice la **plantilla ASP.NET Core WEB API** con el framework **.NET 8**. Asegúrese que la opción Habilitar compatibilidad con el contenedor este **DESHABILITADA**, esto se debe a que el Docker file se agregara manualmente.



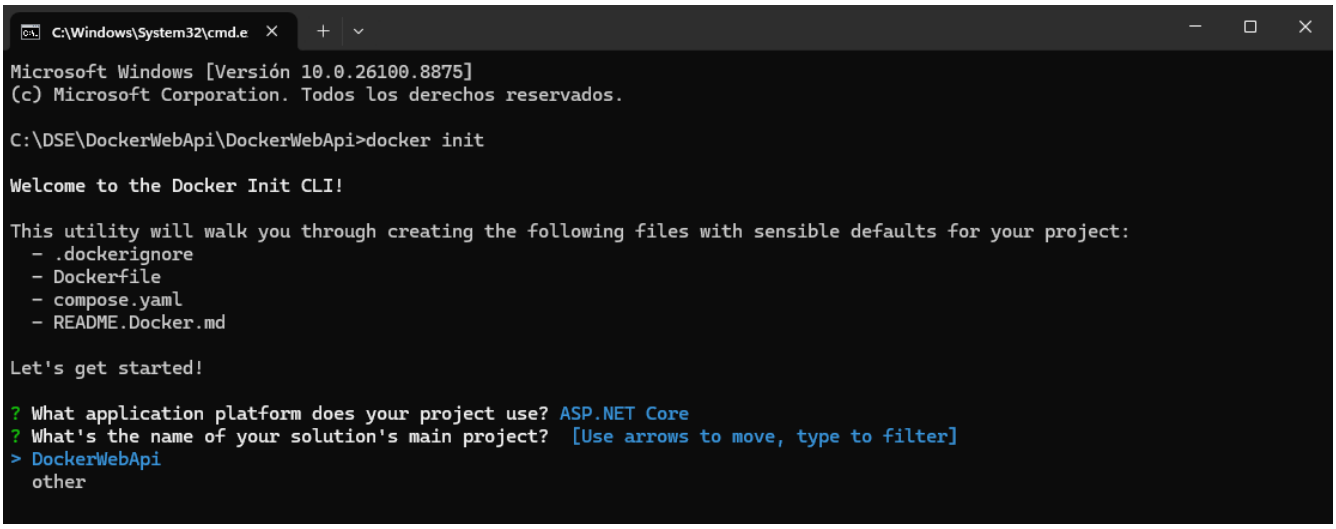


2. Abra una nueva terminal de su preferencia en la carpeta raíz del proyecto y escriba el comando **docker init**. Luego, utilizando las teclas ↑ **Arriba** o ↓ **Debajo** de su teclado, seleccione la plataforma de su proyecto **ASP.NET Core**, es posible que se haya detectado automáticamente por Docker, posteriormente presione la tecla **Enter** para confirmar la selección.

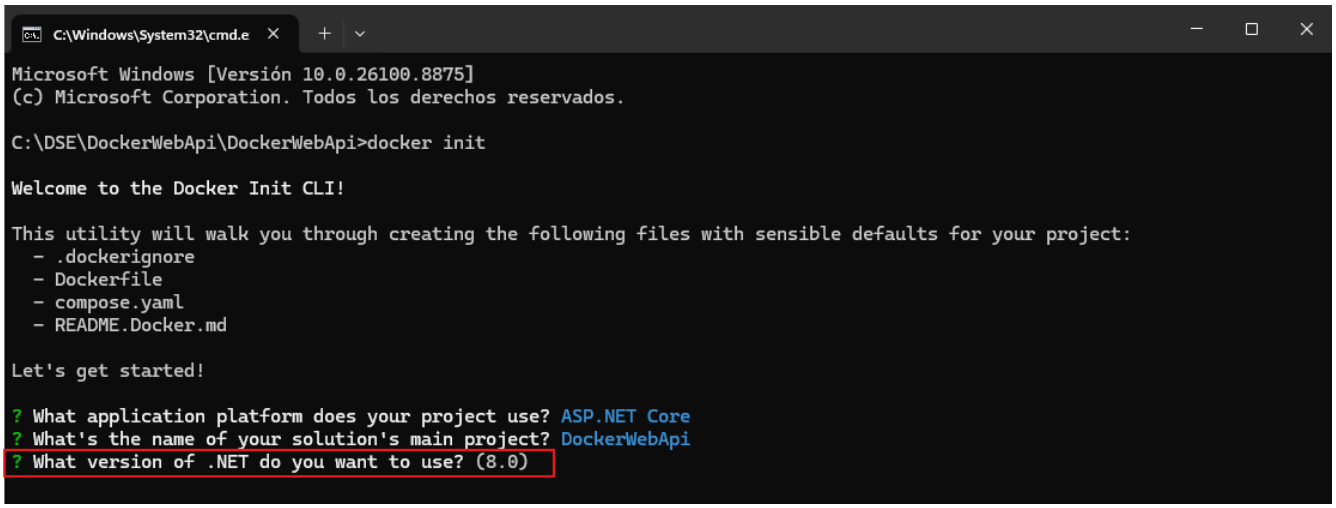




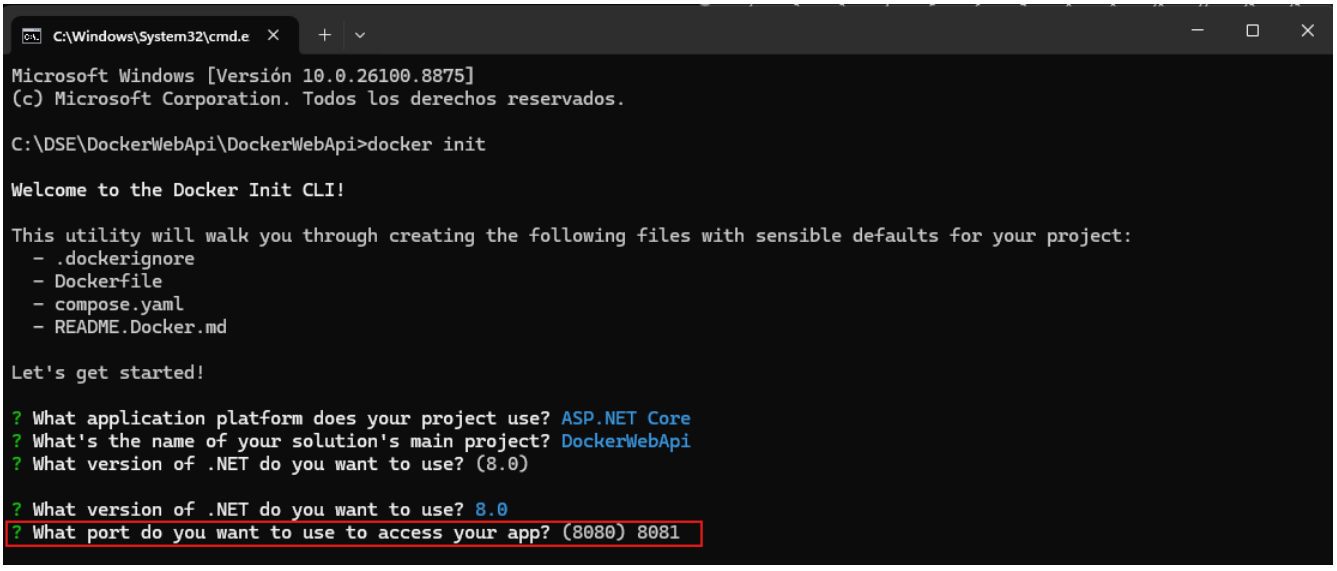
3. Seleccione el nombre de la solución **DockerWebApi** y presione la tecla **Enter**.



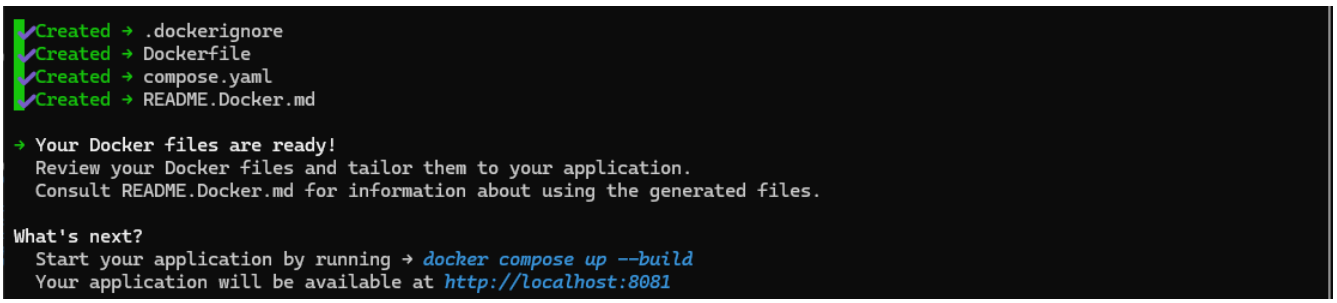
4. Escriba la versión de .NET de la solución, Docker puede detectarla automáticamente, si no es el caso escriba la versión correspondiente y presione la tecla **Enter**.



5. Por último, asigne el puerto que debe exponer el contenedor, por defecto se sugiere el puerto 8080, tenga en cuenta que es un puerto 8080 es un puerto genérico y utilizado frecuentemente como alternativa al puerto 80 para servicios web HTTP. Para este ejemplo asigne el puerto **8081** para evitar conflictos con otra aplicación. Una vez asignado el puerto presione la tecla **Enter**.



6. Una vez finalizado el proceso, observe en el explorador de soluciones que se creación cuatro nuevos archivos.



- **.dockerignore**: indica qué archivos y carpetas no deben incluirse en el contexto de construcción de una imagen Docker.
- **compose.yml**: define uno o varios servicios, redes y volúmenes para ejecutar una aplicación con Docker Compose.
- **README.Docker.md**: es un archivo de documentación específico para Docker, suele contener instrucciones para construir la imagen, iniciar los contenedores, configurar variables de entorno, resolver problemas frecuentes y otros aspectos relacionados con el despliegue o el desarrollo.

7. Explore el Docker File generado.

- **Sintaxis del Dockerfile**

```
# syntax=docker/dockerfile:1
```

Indica la versión de la sintaxis del Dockerfile que utilizará Docker BuildKit, esto permitirá la utilización de características modernas.

- **Build**

```
# Create a stage for building the application.
FROM --platform=$BUILDPLATFORM mcr.microsoft.com/dotnet/sdk:8.0-alpine AS build
```

Esta etapa le conoce como build, y es donde se obtiene la imagen base para este caso, una distribución de Linux Alpine, además, contiene los complementos necesarios para correr aplicaciones .NET.

Con el parámetro **--platform=$BUILDPLATFORM** se indica la arquitectura sobre la cual se está realizando la compilación.

- **Copiar el proyecto y establecer el directorio de trabajo**

```
COPY . /source

WORKDIR /source
```

Copia todo el contenido del proyecto dentro del contenedor y se establece el directorio **/source** sobre el cual se van a ejecutar los comandos.

- **Publicar la aplicación**

```
RUN --mount=type=cache,id=nuget,target=/root/.nuget/packages \
    dotnet publish -a ${TARGETARCH/amd64/x64} --use-current-runtime --self-contained
false -o /app
```

La etapa de **dotnet publish** prepara la aplicación para producción al restaurar las dependencias, compilar el código, optimizarlo y generar todos los archivos necesarios en la carpeta **/app**. Para acelerar el proceso de construcción, utiliza una caché de paquetes NuGet que evita descargar las dependencias en cada ejecución de **docker build**. Además, adapta la arquitectura de compilación convirtiendo **amd64** a **x64**, que es el formato esperado por .NET, y publica la aplicación usando el runtime correspondiente a la plataforma donde se construye la imagen. Al establecer **--self-contained false**, la aplicación no incluye el runtime de .NET, por lo que depende de una imagen base que ya tenga instalado el ASP.NET Runtime, lo que reduce significativamente el tamaño de la imagen final.

- **Etapa final**

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0-alpine AS final
WORKDIR /app

COPY --from=build /app .

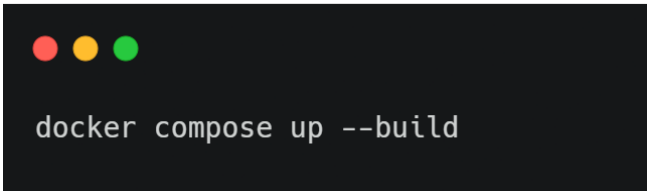
USER $APP_UID

ENTRYPOINT ["dotnet", "DockerWebApi.dll"]
```

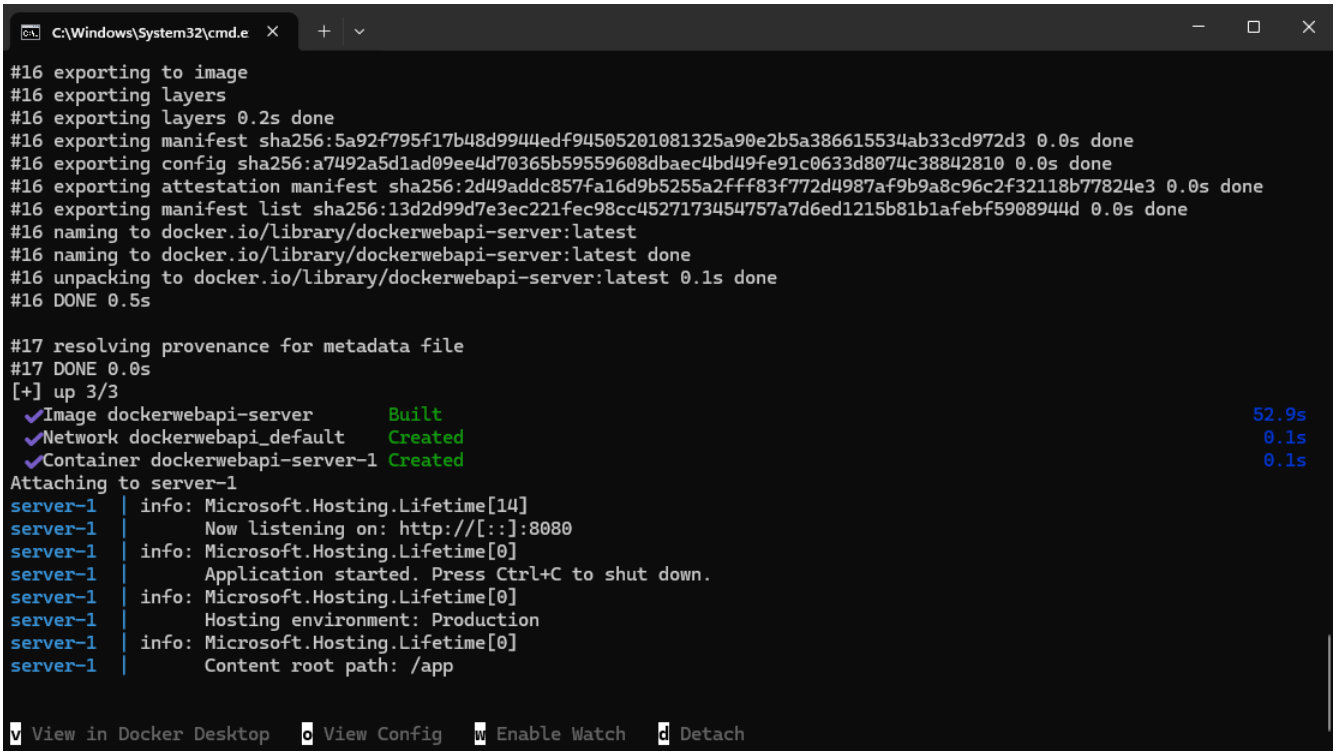
Se utiliza una nueva imagen, pero sin el SDK, únicamente contiene el runtime de ASP.NET y librerías necesarias para ejecutar la aplicación, además, se establece el directorio donde va a vivir la aplicación y se copia el contenido de aplicación compilada.

USER $APP_UID indica que la aplicación se ejecutara con un usuario sin privilegios para mejorar la seguridad, evitando usar root. Luego, **ENTRYPOINT ["dotnet", "DockerWebApi.dll"]** indica el comando que iniciará automáticamente la aplicación al arrancar el contenedor.

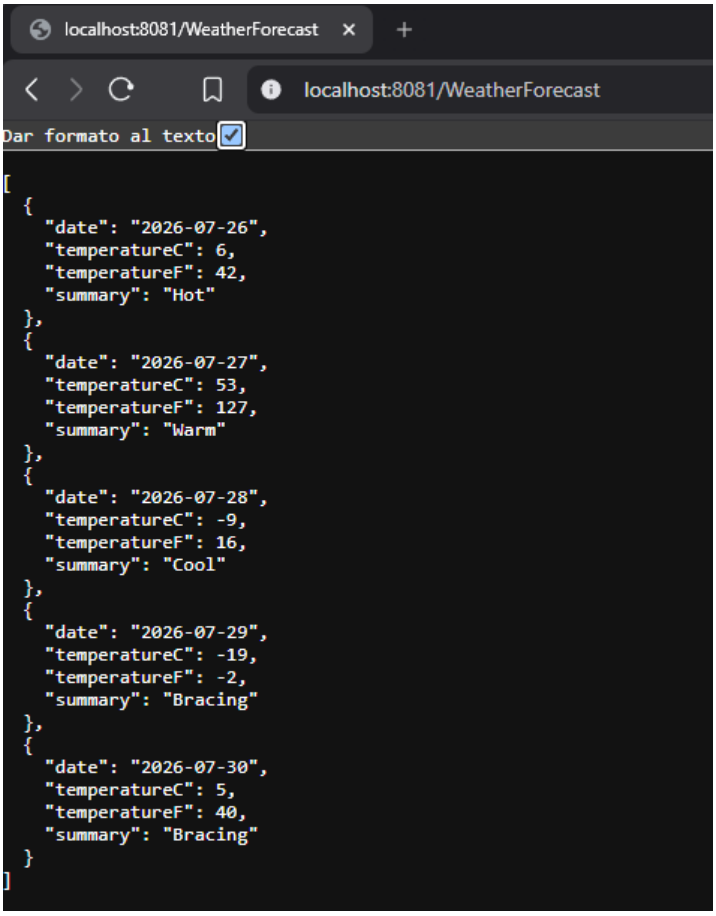
8. En una nueva consola de comandos sobre el directorio raíz del proyecto ejecute el comando **docker compose up --build**.



docker compose up --build permite construir y ejecutar los servicios definidos en un docker file. El comando **docker compose up** crea e inicia los contenedores, redes y volúmenes necesarios, mostrando los logs en la terminal o ejecutándose en segundo plano con la opción **-d**. La opción **--build** fuerza la reconstrucción de las imágenes para aplicar cambios recientes en el código, Dockerfile o configuraciones antes de iniciar los servicios.

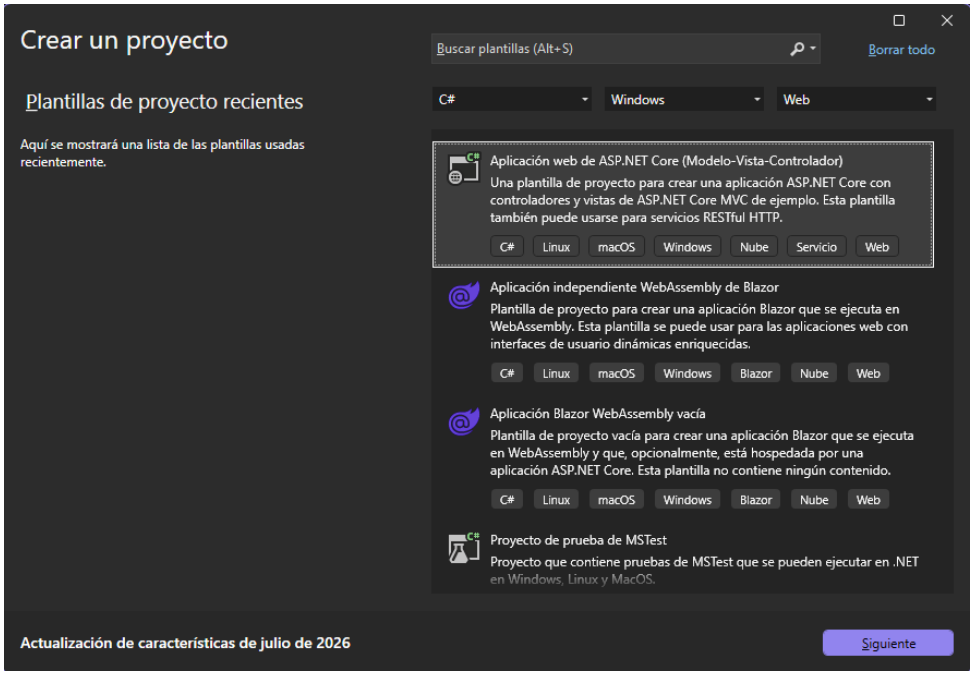


9. En el navegador de su preferencia, compruebe el funcionamiento de la aplicación dockerizada ingresando a la URL http://localhost:8081/WeatherForecast.



Parte 3 – Aplicación MVC con conexión a base de datos

1. Cree un nuevo proyecto ASP.NET MVC con el nombre **DockerMvc** con el framework **.NET 8**. Asegúrese que la opción Habilitar compatibilidad con el contenedor este **DESHABILITADA.**



2. Agregue un nuevo modelo Persona con las siguientes propiedades.

```
namespace DockerMvc.Models;

public class Persona
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Apellido { get; set; } = string.Empty;
    public string NumeroDocumento { get; set; } = string.Empty;

}
```

3. Instale los paquetes NuGet de acuerdo con el nivel de versión de la SDK de .NET.

- Microsoft.EntityFrameworkCore
- Microsoft.EntityFrameworkCore.Design
- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.EntityFrameworkCore.Tools

4. Agregue una nueva carpeta **Data** y cree una nueva clase **ApplicationDbContext** con el siguiente código.

```
using DockerMvc.Models;
using Microsoft.EntityFrameworkCore;

namespace DockerMvc.Data;
```

```
public class ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) :
DbContext(options)
{
    public DbSet<Persona> Personas { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
    }
}
```

5. Configure la cadena de conexión en el archivo **appsettings.json**, utilice una base local para probar el funcionamiento de nuestra aplicación, posteriormente se cambiará la configuración al contenedor.

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=DockerMVC;Trusted_Connec-
tion=True;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

6. Configure la clase **Program** para incluir el contexto de la base de datos.

```
using DockerMvc.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<ApplicationDbContext>(
    options => options.UseSqlServer(builder.Configuration.GetConnectionString("De-
faultConnection")) // <- NUEVO
);

var app = builder.Build();
```

7. Siempre en la clase **Program**, agregue el siguiente fragmento de código para ejecutar todas las migraciones pendientes de aplicar. Asegúrese que el código este antes de la instrucción **app.Run()**.

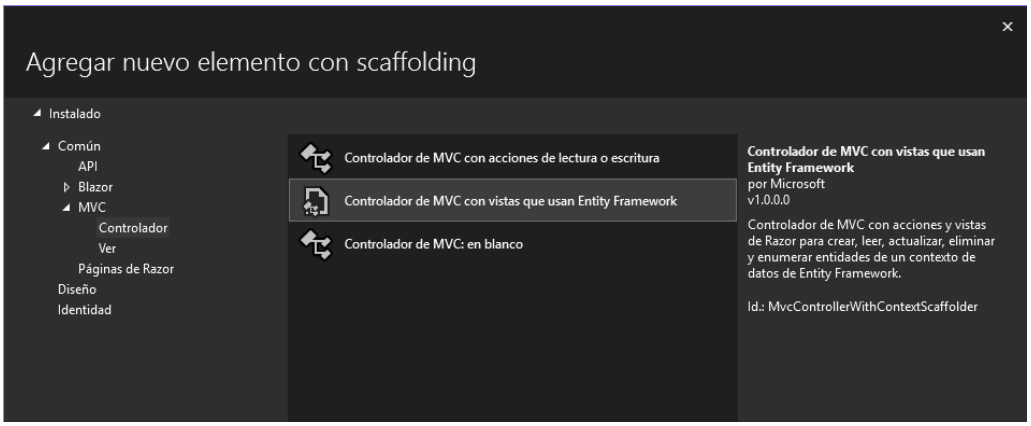
```
using (var scope = app.Services.CreateScope())
{
    var serviceProvider = scope.ServiceProvider;

    var context = serviceProvider.GetRequiredService<ApplicationDbContext>();

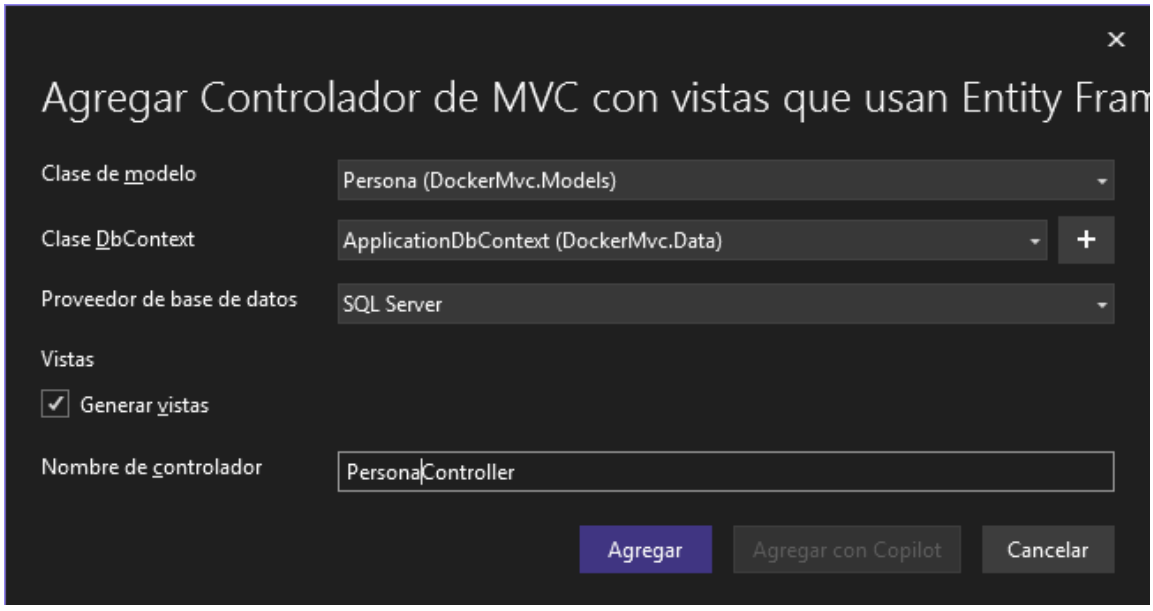
    if (context.Database.GetPendingMigrations().Any())
    {
        context.Database.Migrate();
    }
}

app.Run();
```

8. Agregue un nuevo controlador utilizando Scaffolding.



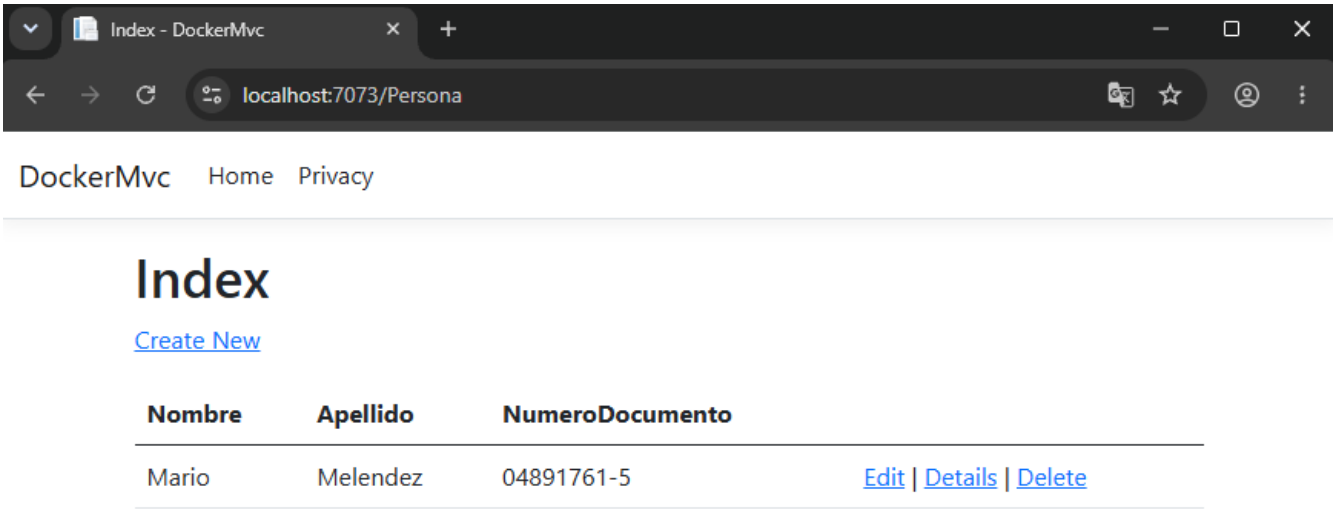
9. Configure el controlador con los parámetros que se muestran a continuación.



10. Agregue una nueva migración con el nombre **Inicial**, asegúrese de NO ejecutar el comando para actualizar la base de datos. Esta acción se realizará de manera automática cuando se ejecute la aplicación.

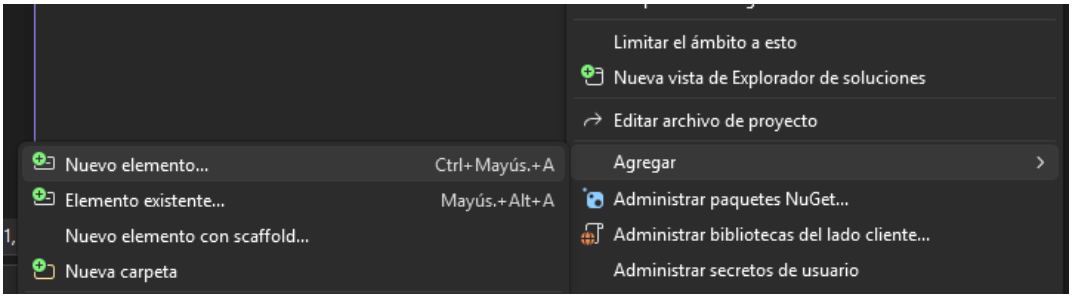
```
PM> Add-Migration Inicial
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM>
```

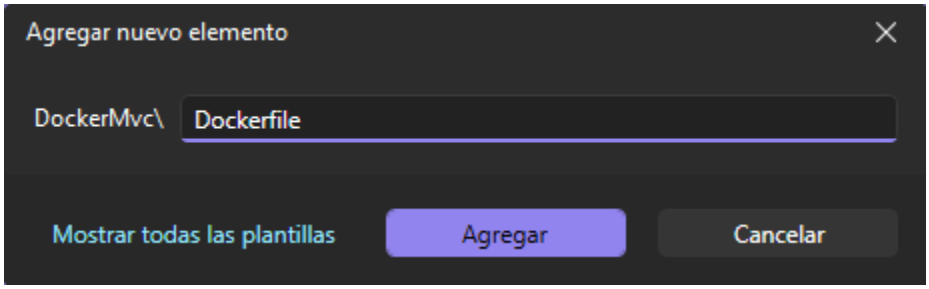
11. Compruebe el funcionamiento de la aplicación agregando nuevos registros.



Parte 4 – Multi contenedores

1. Agregue un nuevo archivo en la raíz del proyecto con el nombre **Dockerfile** en la opción **Agregar > Nuevo elemento**, asegúrese se colocar ninguna extensión.





2. Modifique el código del archivo Dockerfile como se muestra a continuación.

```
# Compilación del proyecto
FROM mcr.microsoft.com/dotnet/sdk:8.0-alpine AS build

ARG BUILD_CONFIGURATION=Release

WORKDIR /src

COPY ["DockerMvc.csproj", "."]

RUN dotnet restore "./DockerMvc.csproj"

COPY . .

RUN dotnet build "./DockerMvc.csproj" -c $BUILD_CONFIGURATION -o /app/build

# Publicar el proyecto
FROM build AS publish

ARG BUILD_CONFIGURATION=Release

RUN dotnet publish "./DockerMvc.csproj" \
    -c $BUILD_CONFIGURATION \
    -o /app/publish \
    /p:UseAppHost=false

# Imagen final para ejecutar
FROM mcr.microsoft.com/dotnet/aspnet:8.0-alpine AS final

WORKDIR /app

COPY --from=publish /app/publish .

ENTRYPOINT ["dotnet", "DockerMvc.dll"]
```

3. Ya que la aplicación necesita una base de datos, esta estará en un nuevo contenedor. Para ello, se definirá un archivo **docker-compose.yml** para administrar los múltiples contenedores. En la raíz del proyecto agregue un nuevo archivo con el nombre **docker-compose.yml**, modifique el contenido como se muestra a continuación.

```
services:
  # Contenedor de la app MVC
  mvc-app:
    build:
      context: .
    ports:
      - "8081:8080"
    depends_on:
      - db # Espera a que inicie la base de datos
```

```
  # Contenedor de la base de datos
  db:
    image: mcr.microsoft.com/mssql/server:2022-latest
    environment:
      SA_PASSWORD: "Admin123" # password del usuario sa
      ACCEPT_EULA: "Y"
    ports:
      - "1433:1433"
    volumes:
      - sqlserver-data:/var/opt/mssql

volumes:
  sqlserver-data:
```

4. Modifique el archivo **appsettings.json** para establecer la cadena de conexión a la base de datos dentro del contenedor.

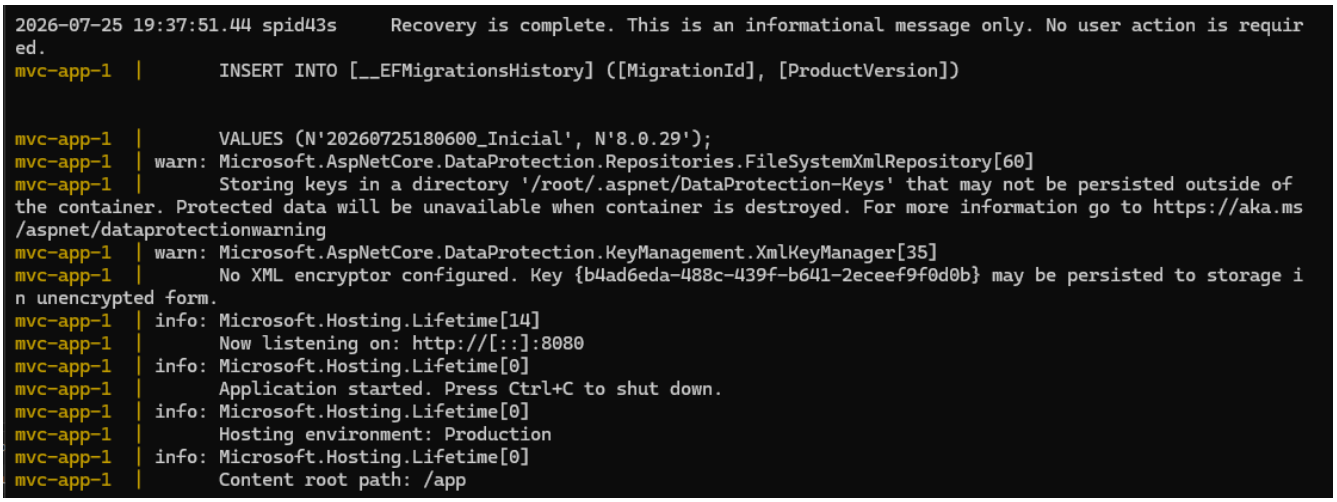
```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=db;Database=DockerMVC;User Id=sa;Password=Admin123;TrustServerCertificate=True;MultipleActiveResultSets=True;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

5. Inicie una nueva terminal a nivel del archivo **DockerMvc.csproj** y ejecute el comando **docker compose up –build**.

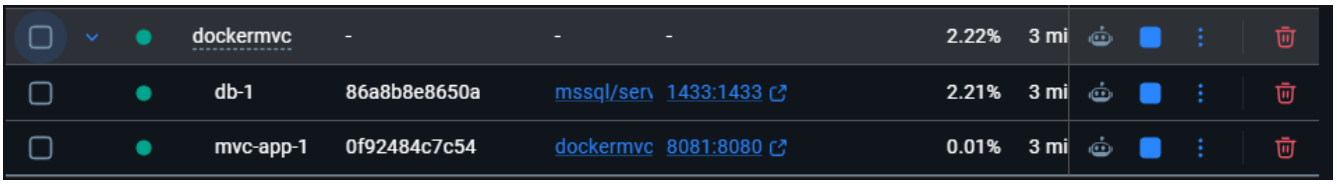
```
docker compose up --build
```



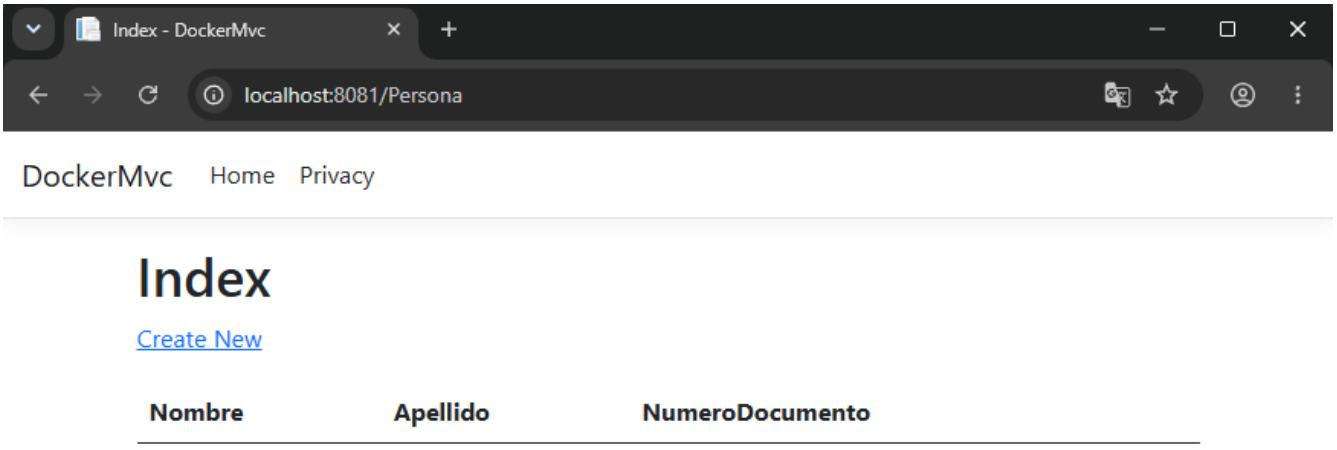
6. Observe en el log que se ejecutó el código para aplicar las migraciones pendientes.



7. Si la construcción de los contenedores fue exitosa, también serán visibles en la aplicación Docker Desktop.



8. Compruebe el funcionamiento de la aplicación corriendo desde el contenedor ingresando a la URL



V. DESARROLLO DE HABILIDADES

Ejercicio 1

Desarrolle una aplicación en ASP .Net Core para gestionar una biblioteca. La aplicación debe manejar información sobre Libros, Autores, y Categorías de libros, con las siguientes especificaciones y relaciones:

- Un libro pertenece a un autor y a una categoría.
- Un autor puede tener varios libros.
- Una categoría puede contener varios libros.

Campos mínimos requeridos

1. **Autores**:
   - Id (entero, clave primaria, auto-incremental)
   - Nombre (cadena, no nulo, longitud máxima 100 caracteres)
   - Apellido (cadena, no nulo, longitud máxima 100 caracteres)

2. **Categorías**:
   - Id (entero, clave primaria, auto-incremental)
   - Nombre (cadena, no nulo, longitud máxima 100 caracteres)

3. **Libros**:
   - Id (entero, clave primaria, auto-incremental)
   - Titulo (cadena, no nulo, longitud máxima 200 caracteres)
   - FechaPublicacion (fecha, no nulo)
   - AutorId (entero, clave foránea que referencia a Autores)
   - CategoriaId (entero, clave foránea que referencia a Categorías)